



DRY Patterns for API Calls in React



Vishal Sharma · Follow

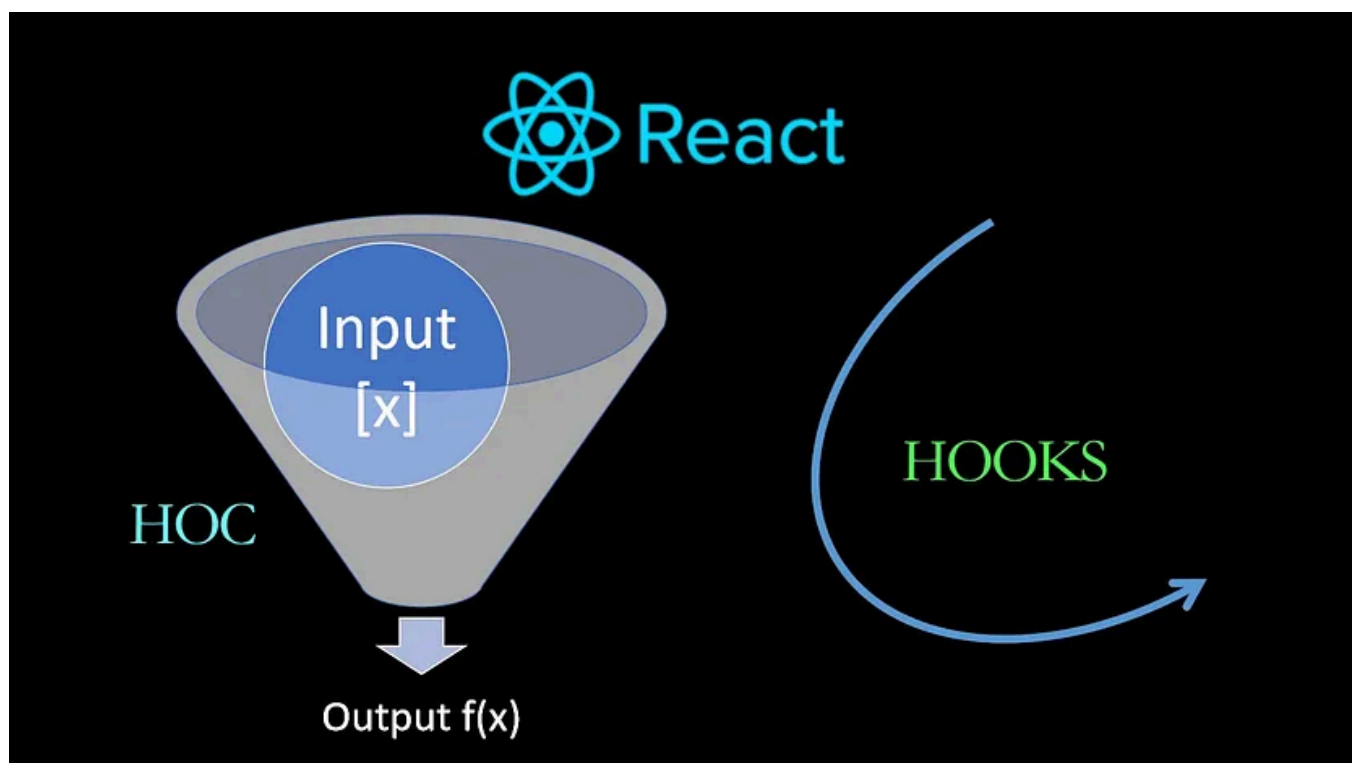
5 min read · Nov 5, 2021



Listen



Share



In most of the React projects, we have components where we need to fetch the data from an API on the mount. React provides us with lifecycle methods, and hooks to accomplish that but usually, we end up duplicating the same code for handling the API call states in all of those components. In this article, we will talk about how can we avoid that by following DRY code principles using different React constructs.

• • •

The Problem

Each API call in an application has three states:

- **Loading** — When the call is made.
- **Successful** — When the API returns a successful response.
- **Error** — When the call fails.

Let's imagine that we are creating a very basic e-commerce application that has a `Catalog` page that shows the list of products available and a `Cart` page that shows the products added to the cart by the user. Both of these pages need to fetch the data from an API on the mount.

Let's get started by creating the API calls to fetch the products and the cart.

```
service.js

export const getProducts = async () => {
  try {
    const response = await fetch('https://mydomain.com/products');
    return response.json();
  }
  catch {
    throw new Error('could not fetch the products');
  }
}

export const getMyCart = async () => {
  try {
    const response = await fetch('https://mydomain.com/cart', {
      header: {
        authToken: 'AuthToken to identify the user'
      }
    });
    return response.json();
  }
  catch {
    throw new Error('could not fetch the cart');
  }
}
```

React class-based components give a lifecycle method named `componentDidMount` to execute the code only once after the initial render. This makes the method best suited for API calls on the mount.

Using the same method, we can create the Catalog and Cart Components as:

```
import React, { Component } from "react";
import { getProducts } from "../service";

class Catalog extends Component {
  state = {
    loading: false,
    data: null,
    error: null,
  }

  componentDidMount() {
    this.setState({ loading: true });
    getProducts()
      .then((data) => {
        this.setState({ data })
      })
      .catch((error) => {
        this.setState({ error })
      })
      .finally(() => {
        this.setState({ loading: false });
      })
  }

  render() {
    const { loading, data, error } = this.state;

    return (
      <>
        {
          loading ?
            <h1>loading...</h1>
          :
          error ?
            <h1>Something went wrong. Please try again later</h1>
          :
            <ul>
              {
                data?.map(d => (
                  <li key={d.id}>{d.title}</li>
                ))
              }
            </ul>
        }
      </>
    )
  }
}
```

```
    </>  
  )  
}  
};  
  
export default Catalog;
```

```
import React, { Component } from "react";
import { getMyCart } from "../service";

class Cart extends Component {
  state = {
    loading: false,
    data: null,
    error: null,
  }

  componentDidMount() {
    this.setState({ loading: true });
    getMyCart()
      .then((data) => {
        this.setState({ data })
      })
      .catch((error) => {
        this.setState({ error })
      })
      .finally(() => {
        this.setState({ loading: false });
      })
  }

  render() {
    const { loading, data, error } = this.state;

    return (
      <>
        {
          loading ?
            <h1>loading...</h1>
          :
          error ?
            <h1>Something went wrong. Please try again later</h1>
          :
            <ul>
              {
                data?.map(d => (
                  <li key={d.id}>{d.title}</li>
                ))
              }
            </ul>
        }
      </>
    )
  }
}
```

```
    </>
  )
}
};

export default Cart;
```

Using Functional Components

React@16.8 came up with the concept of `useState` a hook using which we can maintain the state in functional components as well. Similar to `componentDidMount`, `useEffect` hook with empty dependencies array can be used to make an API call only once after the initial render.

Thus by using the functional components, we can create the Catalog and Cart Components as:

```
import React, { useState, useEffect } from "react";
import { getProducts } from "../service";

const Catalog = () => {
  const [loading, setLoading] = useState(false);
  const [data, setData] = useState(null);
  const [error, setError] = useState(null);

  useEffect(() => {
    setLoading(true);
    getProducts()
      .then((data) => {
        setData(data);
      })
      .catch((error) => {
        setError(error);
      })
      .finally(() => {
        setLoading(false);
      })
  }, []);

  return (
    <>
      {
        loading ?
          <h1>loading...</h1>
          :
          error ?
            <h1>Something went wrong. Please try again later</h1>
            :
            <ul>
              {
                data?.map(d => (
                  <li key={d.id}>{d.title}</li>
                ))
              }
            </ul>
          :
      }
    </>
  );
}
```



```
}
```

```
export default Catalog;
```

```
import React, { useState, useEffect } from "react";
import { getMyCart } from "../service";

const Cart = () => {
  const [loading, setLoading] = useState(false);
  const [data, setData] = useState(null);
  const [error, setError] = useState(null);

  useEffect(() => {
    setLoading(true);
    getMyCart()
      .then((data) => {
        setData(data);
      })
      .catch((error) => {
        setError(error);
      })
      .finally(() => {
        setLoading(false);
      })
  }, []);

  return (
    <>
      {
        loading ?
          <h1>loading...</h1>
        :
        error ?
          <h1>Something went wrong. Please try again later</h1>
        :
          <ul>
            {
              data?.map(d => (
                <li key={d.id}>{d.title}</li>
              ))
            }
          </ul>
      }
    </>
  )
}
```

```
export default Cart;
```

More information about react hooks can be found [here](#).

No matter if you are using the class-based components or functional components, we can see a lot of code duplication.

- Both components have duplicated states to handle the API call state.
- `componentDidMount` or `useEffect` hooks have the same code to fetch the data in both the components.
- Both components have duplicated code in the render method to show the user experience depending upon the API state.

Solution

Refactoring using Higher Order Components

The concept of Higher order components is taken from Higher order functions in functional programming. The difference is, that instead of taking a function as an argument and returning a function, it takes a react component as an argument and returns another react component.

Using the concept, we can create a HOC named `withApiCallOnMount` which can take the component needed to make the API call on the mount and the service as arguments and return a component with the updated API states as props.

```

withApiCallOnMount.js

import React from "react";

const apiStatus = {
  loading: 'loading',
  complete: 'complete',
  errored: 'errored'
}

const withApiCallOnMount = (Component, service) => {

  return class extends React.Component {
    constructor(props) {
      super(props);
      this.state = {
        status: apiStatus.loading,
        data: null,
      };
    }
    componentDidMount() {
      service()
        .then((data) => {
          this.setState({ data, status: apiStatus.complete })
        })
        .catch(() => {
          this.setState({ status: apiStatus.errored });
        })
    }

    render() {
      const { status, data } = this.state;

      return (
        <Component {...this.props} loading={status === apiStatus.loading} data={data} error=
{status === apiStatus.errored} />
      )
    }
  }
}

export default withApiCallOnMount;

```

To use the HOC, Catalog and Cart component will change as:

```
import React from "react";
import { getProducts } from "../service";
import withApiCallOnMount from "../withApiCallOnMount";

const Catalog = ({ loading, data, error }) => {
  return (
    <>
      {
        loading ?
          <h1>loading...</h1>
          :
          error ?
            <h1>Something went wrong. Please try again later</h1>
            :
            <ul>
              {
                data?.map(d => (
                  <li key={d.id}>{d.title}</li>
                ))
              }
            </ul>
          }
      </>
    )
  }
}

export default withApiCallOnMount(Catalog, getProducts);
```

```

Cart.js

import React from "react";
import { getMyCart } from "../service";
import withApiCallOnMount from "../withApiCallOnMount";

const Cart = ({ loading, data, error }) => {
  return (
    <>
      {
        loading ?
        <h1>loading...</h1>
        :
        error ?
        <h1>Something went wrong. Please try again later</h1>
        :
        <ul>
          {
            data?.map(d => (
              <li key={d.id}>{d.title}</li>
            ))
          }
        </ul>
      }
    </>
  )
};

export default withApiCallOnMount(Cart, getMyCart);

```

If we notice, both the components are changed to stateless functional components now as the state logic is moved to a single HOC.

More information about HOCs can be found [here](#)

Refactoring using Custom Hooks

We can use the concept of React hooks to create custom hooks to extract the common code. A custom hook can use the predefined hooks and thus the consumer

component can directly use the custom hooks instead of repeating the predefined hooks.

Using the concept, we can create a custom hook named `useApiCallOnMount` which will take service as an argument and return the API states to the consumer component.

```
useApiCallOnMount.js

import { useState, useEffect } from "react";

const apiStatus = {
  loading: 'loading',
  complete: 'complete',
  errored: 'errored'
}

const useApiCallOnMount = (service) => {
  const [status, setStatus] = useState(apiStatus.loading);
  const [data, setData] = useState(null);

  useEffect(() => {
    service()
      .then((data) => {
        setData(data);
        setStatus(status.complete);
      })
      .catch(() => {
        setStatus(status.errored);
      })
  }, []);

  return [status === apiStatus.loading, data, status === apiStatus.errored];
}

export default useApiCallOnMount;
```

To use the custom hook, the Catalog and Cart component will change as:

```
import React from "react";
import { getProducts } from "../service";
import useApiCallOnMount from "../useApiCallOnMount";

const Catalog = () => {
  const [loading, data, error] = useApiCallOnMount(getProducts);

  return (
    <>
      {
        loading ?
          <h1>loading...</h1>
          :
          error ?
            <h1>Something went wrong. Please try again later</h1>
            :
            <ul>
              {
                data?.map(d => (
                  <li key={d.id}>{d.title}</li>
                ))
              }
            </ul>
          }
      </>
    )
  }

export default Catalog;
```



```
import React from "react";
import { getMyCart } from "../service";
import useApiCallOnMount from "../useApiCallOnMount";

const Cart = () => {
  const [loading, data, error] = useApiCallOnMount(getMyCart);

  return (
    <>
      {
        loading ?
        <h1>loading...</h1>
        :
        error ?
        <h1>Something went wrong. Please try again later</h1>
        :
        <ul>
          {
            data?.map(d => (
              <li key={d.id}>{d.title}</li>
            ))
          }
        </ul>
      }
    </>
  )
}

export default Cart;
```

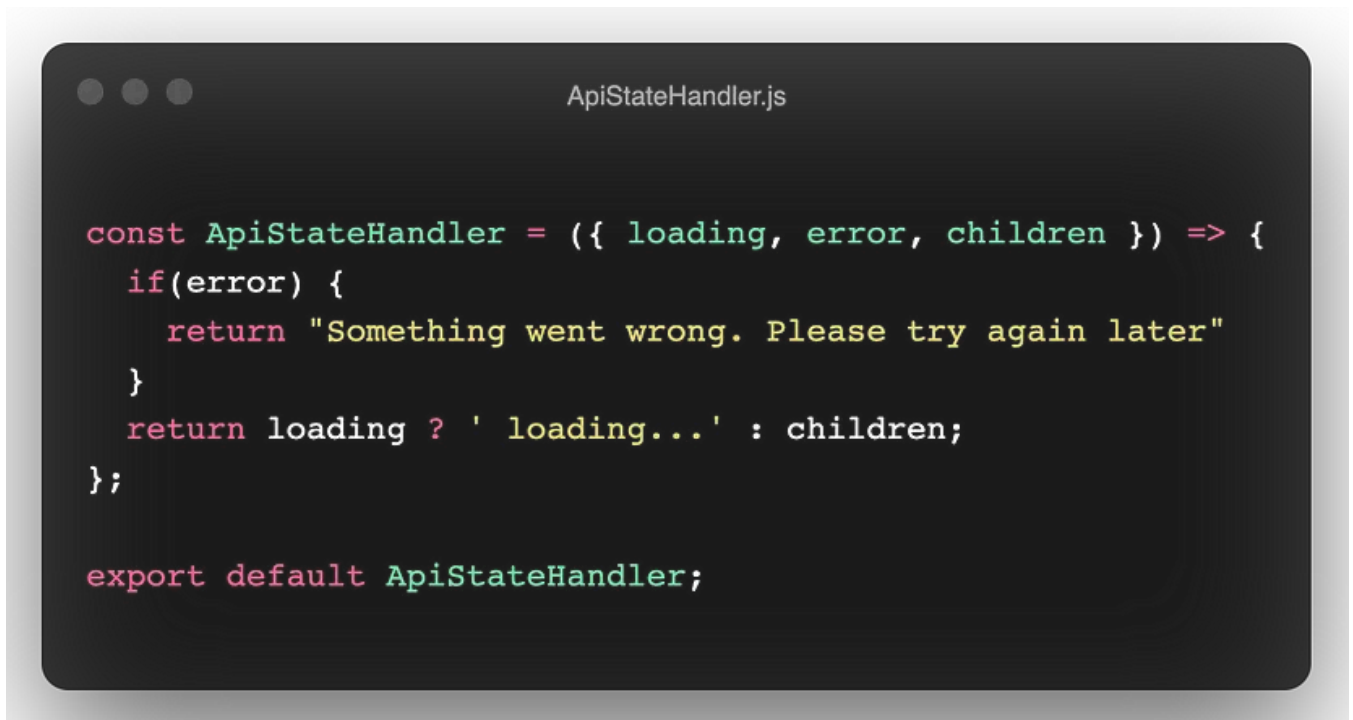
More information about custom hooks can be found [here](#).

In both the above two refactorings using HOC and Custom hook, we have also solved the problem of contradicting states. More information about the same can be found [here](#).

ApiStateHandler as a reusable component

The last thing we need to worry about is the duplicated code in the render method. We are writing the same stuff in both the components to return the JSX according to the API state.

To resolve this, we can create another react component, which takes the API states as props and render the appropriate JSX according to the passed prop.



```
const ApiStateHandler = ({ loading, error, children }) => {
  if(error) {
    return "Something went wrong. Please try again later"
  }
  return loading ? 'loading...' : children;
};

export default ApiStateHandler;
```

To use the ApiStateHandler component, Catalog and Cart components will change as:

```
import React from "react";
import { getProducts } from "../service";
import useApiCallOnMount from "../useApiCallOnMount";
import ApiStateHandler from "../ApiStateHandler";

const Catalog = () => {
  const [loading, data, error] = useApiCallOnMount(getProducts);

  return (
    <ApiStateHandler loading={loading} error={error}>
      <>
        {
          data?.map(d => (
            <li key={d.id}>{d.title}</li>
          ))
        }
      </>
    </ApiStateHandler>
  )
}

export default Catalog;
```

Cart.js

```
import React from "react";
import { getMyCart } from "../service";
import useApiCallOnMount from "../useApiCallOnMount";
import ApiStateHandler from "../ApiStateHandler";

const Cart = () => {
  const [loading, data, error] = useApiCallOnMount(getMyCart);

  return (
    <ApiStateHandler loading={loading} error={error}>
      <>
        {
          data?.map(d => (
            <li key={d.id}>{d.title}</li>
          ))
        }
      </>
    </ApiStateHandler>
  )
}

export default Cart;
```

. . .

In the end, if your application needs more than a simple solution to fetch and update but even caching the response and many more, you can always try the well-known libraries [React Query](#) or [SWR](#) as one in all solutions for you.

A lot more patterns of refactoring a react app can be found in [this Github repo](#).

Higher Order Components

Api Call In React

Dry Code

React Custom Hooks

React Api State Handling



Follow



Written by Vishal Sharma

129 Followers

Front End Engineer at /thoughtworks.

More from Vishal Sharma



Tennis Shoes

129.95 **99.95**

ADD TO CART



Running Shoes

129.95 **99.95**

ADD TO CART

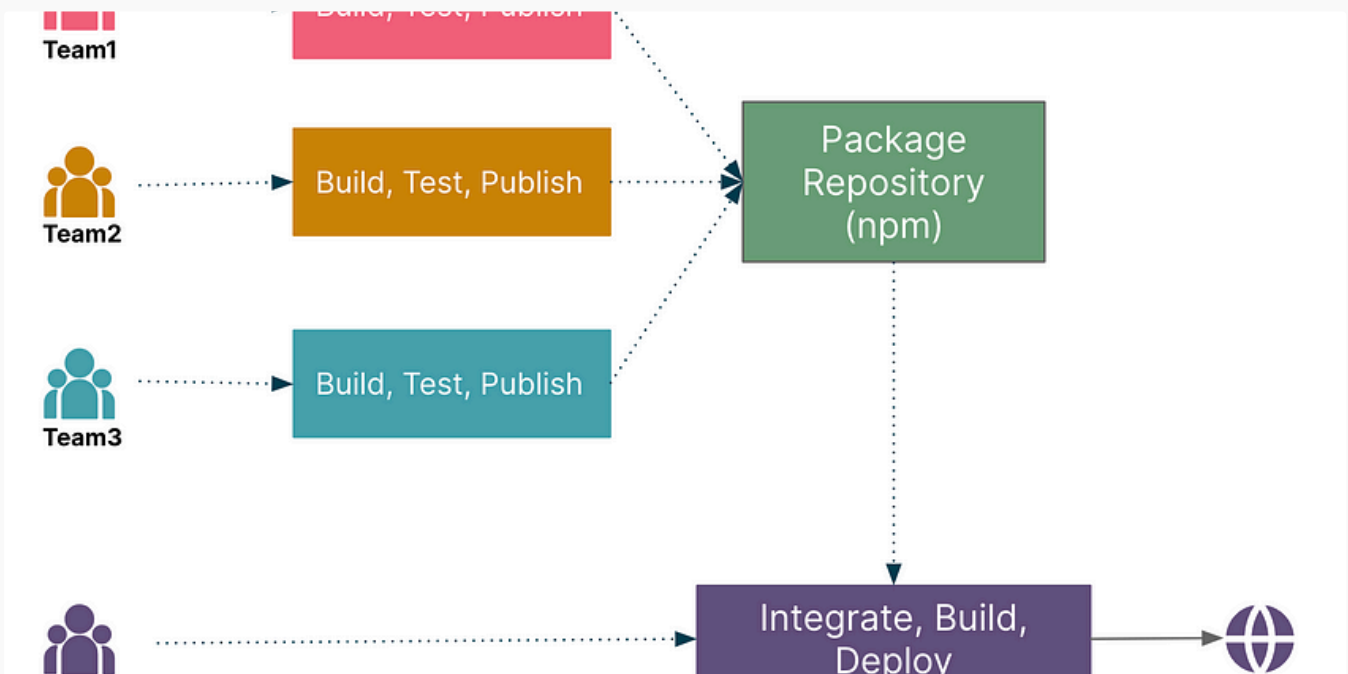


Vishal Sharma

5 Different Techniques for Cross Micro Frontend Communication

The title itself signifies what we are going to discuss in this article. But before we even start, let's try to understand why cross-micro...

Jun 12, 2022 🖱️ 247 💬 4



Vishal Sharma

How to Compose Micro Frontends at Build Time

We have been building npm packages and using those inside the frontend application for eternity now. Build time composition of Micro...



```
package.json

{
  "name": "ts-eslint-jest-setup",
  "version": "1.0.0",
  "main": "index.js",
  "license": "MIT"
}
```

 Vishal Sharma

Setting up Typescript + EsLint + Jest Project

A lot of node projects have a setup which usually include Typescript for static type checking, esLint for static code analysis and Jest as...



 Vishal Sharma

SSR and Web Components— A Contentious Couple

Usually, we talk about the technologies that work best together. This article is the exact opposite where I would like to discuss the...

Jul 31 🖱 9



See all from Vishal Sharma

Recommended from Medium

Amazon.com

Software Development Engineer

Seattle, WA

Mar. 2020 – May 2021

- Developed Amazon checkout and payment services to handle traffic of 10 Million daily global transactions
- Integrated Iframes for credit cards and bank accounts to secure 80% of all consumer traffic and prevent CSRF, cross-site scripting, and cookie-jacking
- Led Your Transactions implementation for JavaScript front-end framework to showcase consumer transactions and reduce call center costs by \$25 Million
- Recovered Saudi Arabia checkout failure impacting 4000+ customers due to incorrect GET form redirection

Projects

NinjaPrep.io (React)

- Platform to offer coding problem practice with built in code editor and written + video solutions in React
- Utilized Nginx to reverse proxy IP address on Digital Ocean hosts
- Developed using Styled-Components for 95% CSS styling to ensure proper CSS scoping
- Implemented Docker with Seccomp to safely run user submitted code with < 2.2s runtime

HeatMap (JavaScript)

- Visualized Google Takeout location data of location history using Google Maps API and Google Maps heatmap code with React
- Included local file system storage to reliably handle 5mb of location history data
- Implemented Express to include routing between pages and jQuery to parse Google Map and implement heatmap overlay



Alexander Nguyen in Level Up Coding

The resume that got a software engineer a \$300,000 job at Google.

1-page. Well-formatted.



Jun 1



24K



483



New JavaScript features

JS

```
const fruits = [
  { name: 'pineapple🍍', color: '🟡' },
  { name: 'apple🍏', color: '🔴' },
  { name: 'banana🍌', color: '🟡' },
  { name: 'strawberry🍓', color: '🔴' },
];
```

```
// ✅ native group by
const groupedByColor = Object.groupBy(
  fruits
```

```
// data-fetcher.js
// ...
const { promise, resolve, reject } =
Promise.withResolvers();
```

 Tari Ibaba in Coding Beauty

5 amazing new JavaScript features in ES15 (2024)

5 juicy ES15 features with new functionality for cleaner and shorter JavaScript code in 2024.

🌟 Jun 2 🖱️ 3K 💬 20

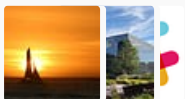


Lists



Staff Picks

750 stories · 1372 saves



Stories to Help You Level-Up at Work

19 stories · 836 saves



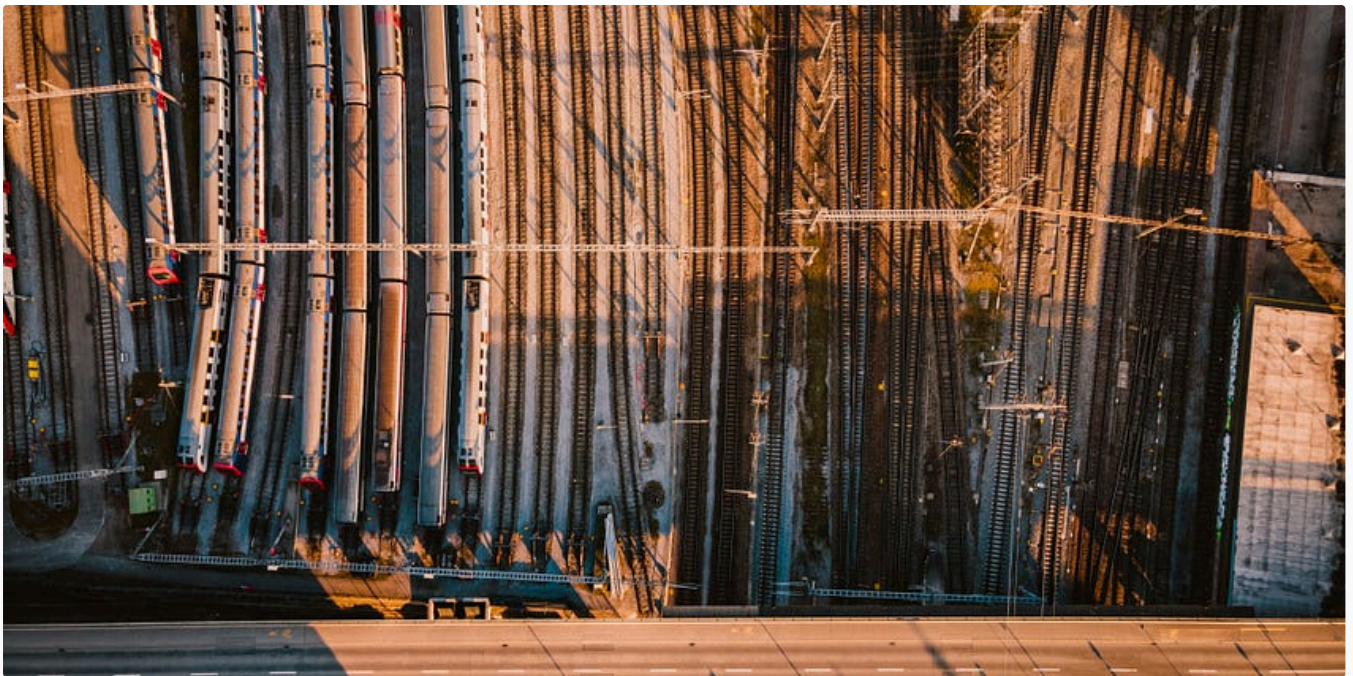
Self-Improvement 101

20 stories · 2869 saves



Productivity 101

20 stories · 2443 saves



 Andrew Zuo

Async Await Is The Worst Thing To Happen To Programming

I recently saw this meme about async and await.

★ Jun 22 🖱 3.8K 💬 221



```
console.log('start');
```

```
const promise1 = new Promise((resolve, reject) => {  
  console.log(1)  
  resolve(2)  
})
```

```
promise1.then(res => {  
  console.log(res)  
})
```

```
console.log('end');
```

start

 Shuai Li in Programming Domain

Can You Answer This Senior Level JavaScript Promise Interview Question?

Most interviewees failed on it.

★ Aug 12 🖱️ 3.1K 💬 33



 Afan Khan in JavaScript in Plain English

Microsoft is ditching React

Here's why Microsoft considers React a mistake for Edge.

★ Jun 6 🖱️ 3.7K 💬 81



 Uyijustindavid

RTK Query vs React/TanStack Query: A Comprehensive Comparison for Frontend Developers.

I. Introduction

Jun 29



See more recommendations